

DeepOHeat-v1: Efficient Operator Learning for Fast and Trustworthy Thermal Simulation and Optimization in 3D-IC Design

Xinling Yu[✉], Ziyue Liu[✉], Hai Li[✉], Yixing Li, Xin Ai, Zhiyu Zeng, Ian Young[✉], *Life Fellow, IEEE*, and Zheng Zhang[✉]

Abstract—Thermal analysis is crucial in 3-D integrated circuit (3D-IC) design due to increased power density and complex heat dissipation paths. Although operator learning frameworks such as DeepOHeat have demonstrated promising preliminary results in accelerating thermal simulation, they face critical limitations in prediction capability for multiscale thermal patterns, training efficiency, and trustworthiness of results during design optimization. This article presents DeepOHeat-v1, an enhanced physics-informed operator learning framework that addresses these challenges through three key innovations. First, we integrate Kolmogorov–Arnold networks (KANs) with learnable activation functions as trunk networks, enabling an adaptive representation of multiscale thermal patterns. This approach achieves a $1.25\times$ and $6.29\times$ reduction in error in two representative test cases. Second, we introduce a separable training method that decomposes the basis function along the coordinate axes, achieving $62\times$ training speedup and $31\times$ GPU memory reduction in our baseline case, and enabling thermal analysis at resolutions previously infeasible due to GPU memory constraints. Third, we propose a confidence score to evaluate the trustworthiness of the predicted results, and further develop a hybrid optimization workflow that combines operator learning with finite difference (FD) using the generalized minimal residual method (GMRES) for incremental solution refinement, enabling efficient and trustworthy thermal optimization. Experimental results demonstrate that DeepOHeat-v1 achieves accuracy comparable to optimization using high-fidelity FD solvers, while speeding up the entire optimization process by $70.6\times$ in our test cases, effectively minimizing the peak temperature through optimal placement of heat-generating components. Open source code is available at <https://github.com/xlyu0127/DeepOHeat-v1>

Index Terms—3-D integrated circuit (3D-IC), Kolmogorov–Arnold networks (KANs), operator learning, thermal simulation.

Received 4 April 2025; revised 2 July 2025 and 26 September 2025; accepted 4 October 2025. Date of publication 9 October 2025; date of current version 21 January 2026. This work was supported in part by the Technology Research (TR) and in part by the Business Development Strategic Research Sector (BD SRS) of Intel Corporation. Recommended for publication by Associate Editor A. Jain upon evaluation of reviewers' comments. (*Corresponding author: Xinling Yu.*)

Xinling Yu and Zheng Zhang are with the Department of Electrical and Computer Engineering, University of California, Santa Barbara, Santa Barbara, CA 93106 USA (e-mail: xyu644@ucsb.edu; zhengzhang@ece.ucsb.edu).

Ziyue Liu is with the Department of Computer Science, University of California, Santa Barbara, Santa Barbara, CA 93106 USA (e-mail: ziyueliu@ucsb.edu).

Hai Li and Ian Young are with Intel Corporation, Hillsboro, OR 97124 USA (e-mail: hai.li@intel.com; ian.young@intel.com).

Yixing Li, Xin Ai, and Zhiyu Zeng are with Cadence Design Systems, Austin, TX 78759 USA (e-mail: yixingli@cadence.com; nathanai@cadence.com; zzeng@cadence.com).

Digital Object Identifier 10.1109/TCPMT.2025.3619906

I. INTRODUCTION

THREE-DIMENSIONAL integrated circuit (3D-IC) is a promising solution to address the scaling challenges of semiconductor chips [2]. Although vertical stacking of multiple active silicon layers enables a higher integration density and improved system performance [3], it also introduces critical thermal management challenges [4], [5], [6]. Heat generated in one layer must pass through multiple layers before reaching the heat sink, creating thermal bottlenecks and potential reliability issues [7]. Addressing these thermal challenges requires extensive analysis and optimization during the design phase. This process involves solving partial differential equations for numerous design configurations, making it computationally intensive. Traditional numerical methods, such as the finite element method and the finite difference (FD) method, while accurate, are too time-consuming for iterative design optimization [6], [8], [9]. This has motivated the development of various surrogate models, including model order reduction [10], [11], and data-driven approaches [12], [13], [14]. Recent advances in multiscale thermal modeling, such as the ROM-based approach by [15], have demonstrated the importance of capturing disparate length scales in 3D-IC thermal analysis.

Neural network-based methods can perform real-time predictions for thermal analysis, but face significant limitations. Data-driven approaches [14], [16], [17] require extensive simulation datasets and lack generalization. Physics-informed neural network-based methods [18] reduce data dependency but are limited to specific geometric parameters. Most existing works either fail to solve 3-D full-chip temperature fields or cannot handle different boundary conditions (BCs) and power distributions. Autoencoder-decoder methods [17], [18] work for 2-D prediction but fail with 3-D volumetric power maps, while graph neural networks [19] still require substantial labeled data despite better spatial modeling.

Recently, operator learning frameworks like DeepOHeat [1] and its variant [20] have shown promise in addressing these limitations. DeepOHeat learns a mapping from design configurations to temperature fields through physics-informed training, eliminating the need for expensive simulation data and effectively handling 3-D geometries. Despite these advances, DeepOHeat faces three critical limitations. First, the multilayer perceptron (MLP) trunk networks exhibit

inherent spectral bias [21], [22], limiting their ability to capture multiscale thermal patterns that are crucial for accurate temperature prediction. Second, its physics-informed training requires computing high-order derivatives across numerous collocation points, making it memory-intensive and time-consuming. This computational burden prevents its application to large-scale problems with high-resolution thermal analysis. Third, like most operator learning frameworks, DeepOHeat lacks mechanisms to assess prediction trustworthiness during optimization, potentially leading to suboptimal or unreliable design decisions.

This article presents *DeepOHeat-v1* to address the above limitations. Through architectural and algorithmic innovations, DeepOHeat-v1 can enhance both computational efficiency and prediction reliability of operator learning for thermal analysis, and enable thermal optimization for 3D-IC floorplanning with both low computational cost and high accuracy. Our key contributions are summarized below.

- 1) Accuracy improvement through Kolmogorov–Arnold networks (KANs). Unlike MLPs with fixed activation functions, KANs employ learnable univariate functions [23], [24], allowing a more adaptive representation of multiscale thermal patterns. We express KAN as DeepOHeat trunk networks, achieving $1.25\times$ and $6.29\times$ error reductions in two representative test cases.
- 2) Efficiency improvement via separable training. We employ the recently developed separable training method [25] to decompose the basis functions of DeepOHeat along coordinate axes. Using forward-mode automatic differentiation (AD) [26], this approach achieves $62\times$ training speedup and $31\times$ memory reduction in our baseline case, making high-resolution thermal analysis computationally feasible.
- 3) Trustworthy thermal optimization. Lack of trustworthiness is a fundamental challenge that prevents the application of almost all neural network-based surrogates in design optimization. We propose a confidence estimator to evaluate whether a DeepOHeat prediction is acceptable or not at every step of an optimization loop. Once a prediction is not acceptable (for some corner design cases), we use it as an initial guess and call generalized minimal residual method (GMRES) (the GMRES [27]) for only a small number of iterations to get a high-fidelity FD solution. This hybrid framework achieves an excellent trade-off between computational efficiency and solution accuracy. We apply this optimization framework to optimize thermal characteristics in 3-D chip designs and obtain an accuracy comparable to that of an FD-based optimizer while achieving $70.6\times$ speedup.

II. BACKGROUND, RELATED WORK, AND MOTIVATION FOR DEEPOHEAT-V1

This section first presents the fundamental PDE governing thermal behavior in 3D-ICs, then reviews how DeepOHeat [1] reformulates this problem through operator learning, highlighting its strengths and limitations. These limitations motivate our proposed enhancements in DeepOHeat-v1.

A. Thermal Simulation in 3D-ICs

Thermal simulation plays a fundamental role in 3D-IC design, where accurate temperature prediction is essential for reliability and performance optimization. The temperature distribution within a chip domain S is governed by the heat conduction equation

$$\frac{\partial}{\partial y_1} \left(k \frac{\partial T}{\partial y_1} \right) + \frac{\partial}{\partial y_2} \left(k \frac{\partial T}{\partial y_2} \right) + \frac{\partial}{\partial y_3} \left(k \frac{\partial T}{\partial y_3} \right) + q_v = \rho c_p \frac{\partial T}{\partial t} \quad (1)$$

where T represents the temperature field and $q_v = q_v(\mathbf{y}, t)$ denotes the volumetric heat generation rate at any spatial-temporal coordinate $(y_1, y_2, y_3, t) \equiv (\mathbf{y}, t)$. The material-specific properties k , ρ , and c_p correspond to thermal conductivity, mass density, and heat capacity, respectively.

In this work, we consider heterogeneous multilayer chip architectures where the thermal conductivity varies vertically across different material layers. Mathematically, this variation is captured by expressing k as a piecewise function of the vertical coordinate: $k = k(y_3)$. Within each layer, we assume isotropic thermal conductivity, meaning that heat conduction is uniform in all directions, i.e. $k_{y_1} = k_{y_2} = k_{y_3} = k(y_3)$.

In this work, we focus on steady-state thermal analysis. Under steady-state conditions ($\partial T / \partial t = 0$), (1) reduces to

$$\nabla \cdot (k(y_3) \nabla T) + q_v(\mathbf{y}) = 0 \quad (2)$$

where $\nabla \cdot (k(y_3) \nabla T)$ accounts for the variation of thermal conductivity across layers.

The thermal behavior at the chip boundaries is governed by appropriate BCs that reflect the thermal management strategy. In this work, we consider three fundamental types of BCs.

- 1) Neumann BC, specifying heat flux at the surface

$$-k(y_3) \frac{\partial T}{\partial y_i} = q_n \quad (3)$$

where q_n represents the local heat flux density. This condition is particularly important for modeling surface power maps in 3D-ICs.

- 2) Adiabatic BC, representing perfectly insulated surfaces

$$\frac{\partial T}{\partial y_i} = 0. \quad (4)$$

- 3) Convection BC, modeling heat exchange with the ambient environment

$$-k(y_3) \cdot \frac{\partial T}{\partial y_i} = h(T - T_{\text{amb}}) \quad (5)$$

where h denotes the heat transfer coefficient and T_{amb} represents the ambient temperature.

B. DeepOHeat for Thermal Analysis

The thermal system described above presents significant computational challenges for design optimization. Although traditional mesh-based numerical methods provide accurate solutions, they are computationally expensive, particularly for iterative design evaluations. To address this challenge, DeepOHeat [1] reformulates thermal simulation through operator learning, enabling rapid temperature prediction without requiring expensive simulation data.

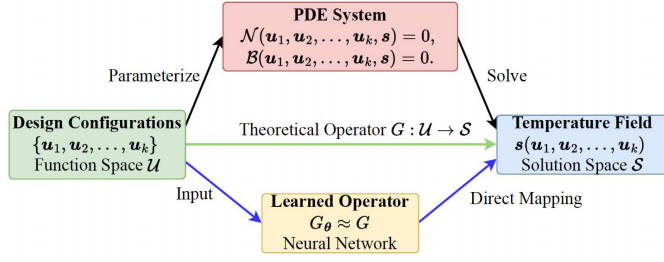


Fig. 1. Operator learning framework for thermal simulation. The theoretical operator G provides a direct mapping from design configurations to temperature fields and is fundamentally defined by the PDE system. The objective of operator learning is to train a surrogate neural operator G_θ that approximates G , enabling rapid temperature prediction without solving PDEs for each new design configuration.

1) Thermal Simulation as an Operator Learning Problem:

As illustrated in Fig. 1, the thermal system governed by (2)–(5) can be formulated in operator form

$$\begin{aligned}\mathcal{N}(\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_k, s) &= 0 \\ \mathcal{B}(\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_k, s) &= 0\end{aligned}\quad (6)$$

where $\mathcal{N}$ represents the heat equation operator, $\mathcal{B}$ denotes the boundary operators, and s is the temperature field. The system is parameterized by k design configurations $\{\mathbf{u}_i\}_{i=1}^k$, which can be either parametric (e.g., heat transfer coefficients) or non-parametric (e.g., power distributions).

In the traditional approach, solving this PDE system requires expensive numerical computations for each new set of design configurations, as shown in the upper path of Fig. 1. Given these PDE configurations $\{\mathbf{u}_i\}_{i=1}^k$, the temperature field evaluated at spatial coordinates $\mathbf{y}$ is denoted as

$$s(\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_k)(\mathbf{y}). \quad (7)$$

The key assumption of the operator learning framework is that there exists a unique operator G that maps the function space $\mathcal{U} = \mathcal{U}_1 \times \mathcal{U}_2 \times \dots \times \mathcal{U}_k$, spanned by the PDE configurations, to the solution space $\mathcal{S}$, such that

$$G : \mathcal{U} \rightarrow \mathcal{S} \quad (8)$$

where $s(\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_k) = G(\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_k)$. The objective of operator learning is to train a surrogate operator model G_θ that approximates G . Once trained, G_θ can instantly predict the temperature distribution for new design configurations, significantly accelerating thermal analysis.

2) *DeepOHeat Framework*: DeepOHeat employs an operator learning architecture based on DeepONet [28], [29] to construct the temperature field approximation, as illustrated in Fig. 2. The framework combines k branch networks that process k design configurations with a trunk network that handles spatial dependencies, enabling efficient learning of the thermal operator mapping. Without loss of generality, we consider the case $k = 1$ to illustrate the core methodology.

As shown in the figure, the temperature field approximation process consists of three key components. First, an encoder $\mathcal{E} : \mathcal{U} \rightarrow \mathbb{R}^m$ maps the input function $\mathbf{u}$ (e.g., surface power) to its point-wise evaluations at m fixed sensor locations $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m$ (e.g., some grid points of a surface), yielding

$(\mathbf{u}(\mathbf{x}_1), \mathbf{u}(\mathbf{x}_2), \dots, \mathbf{u}(\mathbf{x}_m)) = \mathcal{E}(\mathbf{u})$. This encoded representation captures the essential features of the design.

This encoded representation is then processed by two specialized neural networks. The branch network $b_\psi : \mathbb{R}^m \rightarrow \mathbb{R}^r$, parameterized by ψ , transforms the encoded input into r feature coefficients $(\beta_1, \beta_2, \dots, \beta_r)$. Simultaneously, the trunk network $t_\phi : \mathbb{R}^d \rightarrow \mathbb{R}^r$, parameterized by ϕ , maps spatial coordinates $\mathbf{y} = (y_1, y_2, y_3)$ to r basis functions $(\tau_1(\mathbf{y}), \tau_2(\mathbf{y}), \dots, \tau_r(\mathbf{y}))$. The predicted temperature field is then computed as

$$G_\theta(\mathbf{u})(\mathbf{y}) = \sum_{k=1}^r \beta_k \tau_k = b_\psi(\mathcal{E}(\mathbf{u})) \cdot t_\phi(\mathbf{y}) \quad (9)$$

where $\theta = (\psi, \phi)$ encompasses all trainable parameters and $\cdot$ denotes vector dot product.

A key advantage of DeepOHeat, as illustrated on the right side of Fig. 2, is its physics-informed training strategy [30], [31] that eliminates the need for expensive simulation data. Given a set of collocation points $\mathcal{D}$ comprising N_f input designs $\{\mathbf{u}^{(i)}\}_{i=1}^{N_f}$ sampled from $\mathcal{U}$, N_r residual points $\{\mathbf{y}_r^{(j)}\}_{j=1}^{N_r}$ within the chip domain, and N_b boundary points $\{\mathbf{y}_b^{(j)}\}_{j=1}^{N_b}$, the model parameters are optimized by minimizing the physics-informed loss

$$L(\theta|\mathcal{D}) = L_r(\theta|\mathcal{D}) + L_b(\theta|\mathcal{D}) \quad (10)$$

where the residual and boundary losses are defined as

$$\begin{aligned}L_r(\theta|\mathcal{D}) &= \frac{1}{N_f N_r} \sum_{i=1}^{N_f} \sum_{j=1}^{N_r} \left| \mathcal{N}(\mathbf{u}^{(i)}, G_\theta(\mathbf{u}^{(i)})(\mathbf{y}_r^{(j)})) \right|^2 \\ L_b(\theta|\mathcal{D}) &= \frac{1}{N_f N_b} \sum_{i=1}^{N_f} \sum_{j=1}^{N_b} \left| \mathcal{B}(\mathbf{u}^{(i)}, G_\theta(\mathbf{u}^{(i)})(\mathbf{y}_b^{(j)})) \right|^2.\end{aligned} \quad (11)$$

C. Challenges in DeepOHeat for 3D-IC Thermal Analysis

Despite its success in accelerating thermal simulation, DeepOHeat faces three critical limitations that hinder its practical application in thermal-aware design optimization.

First, the accuracy of DeepOHeat heavily depends on the quality of basis functions $\{\tau_k(\mathbf{y})\}_{k=1}^r$ produced by its trunk network t_ϕ . However, the MLP architecture used in the trunk network exhibits inherent spectral bias [21], [22], preferentially learning low-frequency components before high-frequency ones [32]. This limitation significantly impacts DeepOHeat's ability to capture complex thermal patterns with varying spatial frequencies, particularly in regions with sharp temperature gradients or localized hotspots.

Second, although DeepOHeat eliminates the need for simulation data through physics-informed training, this approach introduces significant computational challenges. The physics loss optimization requires computing second-order derivatives with respect to numerous collocation points through reverse-mode AD [33]. For a typical 3-D thermal simulation, discretized on a $101 \times 101 \times 101$ spatial grid, backpropagating through the computational graph across N_f chip configurations and millions of collocation points, becomes prohibitively expensive. This computational burden leads to long training

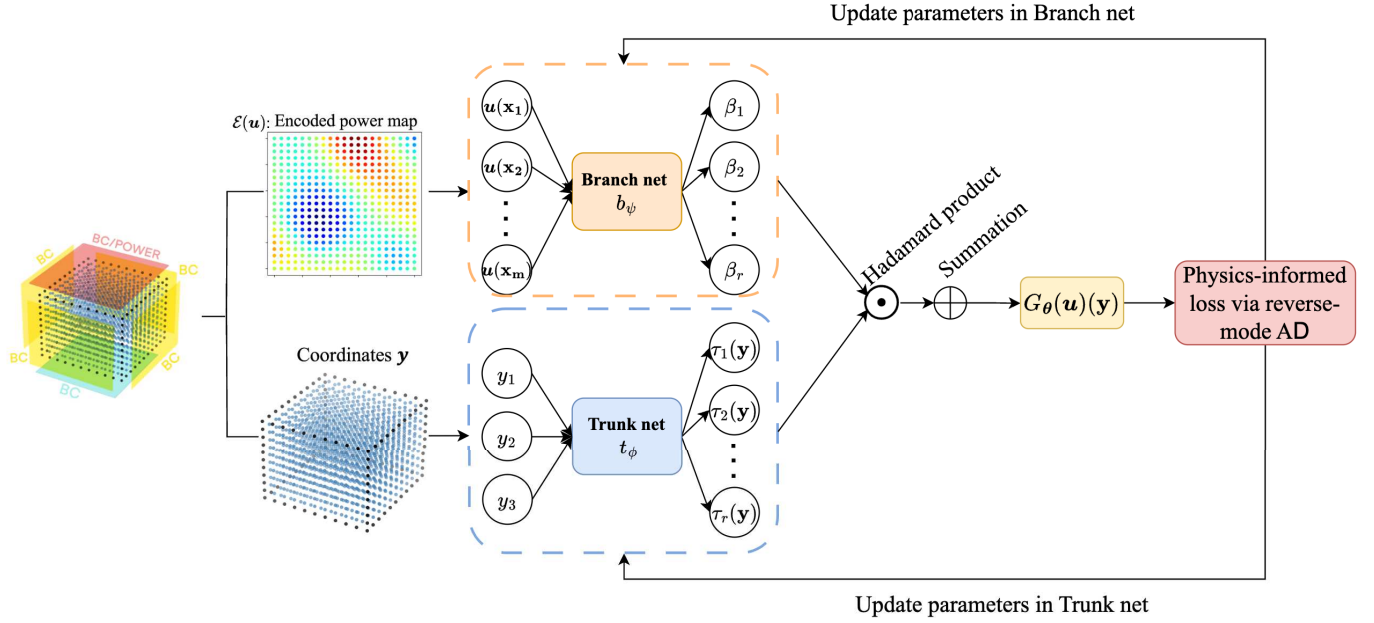


Fig. 2. DeepOHeat for thermal simulation. This example shows the case where $k = 1$, with the surface power map as the design configuration. The architecture consists of branch and trunk networks that process the encoded power map $\mathcal{E}(u)$ and spatial coordinates, respectively. Their outputs are combined through Hadamard product and summation to predict temperature, while physics-informed training with reverse-mode AD enables parameter updates without simulation data.

time and excessive memory cost, preventing DeepOHeat's application to high-resolution thermal analysis.

Finally, while DeepOHeat enables rapid temperature field prediction through (9), it is unclear whether an individual prediction is accurate enough or not. This limitation becomes critical in thermally aware design optimization, where inaccurate predictions could lead to suboptimal or potentially unsafe designs. Without a systematic approach to quantify prediction trustworthiness and refine or correct potentially inaccurate results, the practical deployment of DeepOHeat in automated design workflows remains challenging.

To overcome these fundamental constraints, we propose *DeepOHeat-v1*, which introduces three key enhancements.

- 1) Kolmogorov–Arnold Networks to mitigate the trunk network's spectral bias and accurately capture multiscale thermal patterns.
- 2) A separable training strategy to drastically reduce computational and memory overhead in physics-informed training for high-resolution 3D-IC analysis.
- 3) A hybrid optimization workflow that integrates the operator learning with GMRES-based refinement to ensure both efficiency and trustworthiness during design optimization.

We detail these enhancements in Sections III–V.

III. KANS WITH MULTIFREQUENCY BASIS FUNCTIONS

A major challenge in learning thermal fields is to capture both smooth varying regions and sharply localized hotspots. MLPs often struggle with such multiscale patterns due to spectral bias [21], preferring low-frequency solutions over high-frequency ones [32]. In this context, “multiscale” refers

to thermal fields that simultaneously contain low-frequency spatial components (smooth, gradually-varying regions) and high-frequency components (sharp temperature gradients and localized hotspots). The challenge lies not in having features of vastly different sizes, but in accurately representing both the smooth and sharp frequency characteristics that coexist in thermal solutions, particularly around heat sources and material interfaces where sharp gradients naturally occur even with similar-magnitude heating. To address this, DeepOHeat-v1 uses a KAN [23] as a trunk network. Kolmogorov's representation theorem [34] states that any continuous multivariate function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ can be represented as

$$f(\mathbf{y}) = \sum_{j=1}^{2n+1} g_j \left(\sum_{i=1}^n \phi_{i,j}(y_i) \right) \quad (12)$$

where g_j and $\phi_{i,j}$ are continuous univariate functions, and $\mathbf{y} = (y_1, \dots, y_n) \in \mathbb{R}^n$. This representation suggests constructing complex multivariate functions through compositions of univariate functions.

A. KAN Implementation for Thermal Analysis

In DeepOHeat-v1, we implement a KAN-based trunk network that directly maps spatial coordinates to basis functions $\{\tau_k\}_{k=1}^r$. In Fig. 3, our KAN implementation differs fundamentally from standard MLPs by using learnable univariate functions on the edges rather than fixed activation functions with trainable weights. Each KAN layer transforms inputs as

$$y_j^{(\ell+1)} = \sum_{i=1}^{n_\ell} \phi_{\ell,i,j} \left(y_i^{(\ell)} \right) \quad (13)$$

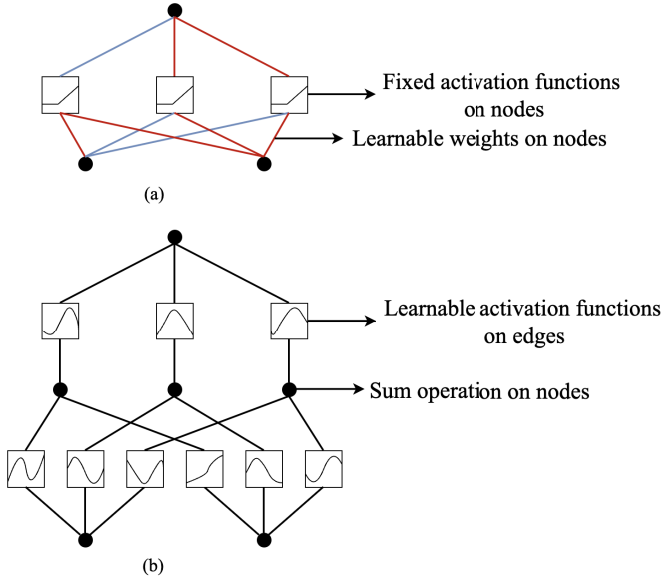


Fig. 3. Comparison between (a) traditional MLP and (b) KAN. In MLPs, the activation functions are fixed at the nodes, while weights are learnable. In contrast, KANs use learnable activation functions on edges and sum operations at nodes, offering greater flexibility in function representation.

where each trainable function $\phi_{\ell,i,j}$ operates on a single variable $y_i^{(\ell)}$ and is parameterized using an order- K Chebyshev polynomial series

$$\phi_{\ell,i,j}(y) = \sum_{k=0}^K a_{\ell,i,j,k} C_k(y) \quad (14)$$

$$C_k(y) = \cos(k \arccos(y)), \quad y \in [-1, 1]$$

where $a_{\ell,i,j,k} \in \mathbb{R}$ are trainable coefficients. This structure inherently supports multiscale representation through its Chebyshev polynomial basis, enabling efficient modeling of both global thermal patterns and localized hotspots. The architecture's ability to naturally represent different frequency scales eliminates the need for explicit Fourier feature mapping [21] or deep architectures typically required in MLPs.

B. Theoretical Understanding of the Training Dynamics

To understand why our KAN implementation effectively captures multiscale thermal patterns, consider a function with a Chebyshev series expansion

$$f^*(y) = \sum_{k=0}^{\infty} b_k C_k(y) \quad (15)$$

where $\{b_k\}$ are the true spectral coefficients. To understand the learning dynamics of KANs with Chebyshev basis functions, consider a simplified single-layer model

$$f(y) = \sum_{k=0}^K a_k C_k(y), \quad y \in [-1, 1] \quad (16)$$

where a_k 's are trainable parameters and $C_k(y)$'s are the Chebyshev polynomials defined in (14). Our network approximates the first $K+1$ components of f^* through gradient descent. Understanding this multifrequency learning process provides insights into the behavior of deeper KAN architectures.

The neural tangent kernel (NTK) characterizes how network outputs change with respect to parameters during training [35], [36]. For our model, the NTK is

$$\Theta_{\text{KAN}}(y, y') = \sum_{k=0}^K \frac{\partial f(y)}{\partial a_k} \frac{\partial f(y')}{\partial a_k} = \sum_{k=0}^K C_k(y) C_k(y'). \quad (17)$$

Under gradient flow (see Appendix B), the output evolves as

$$\frac{\partial f(y, t)}{\partial t} = -\eta \int_{-1}^1 \Theta_{\text{KAN}}(y, y') (f(y', t) - f^*(y')) \frac{dy'}{\sqrt{1-y'^2}} \quad (18)$$

where t is the training time and $\eta > 0$ is the learning rate. The error $e(y, t) = f(y, t) - f^*(y)$ decays spectrally (see Appendix B)

$$e(y, t) = \sum_{k=0}^K e_k(0) e^{-\eta \kappa_k t} C_k(y) \quad (19)$$

where $e_k(0)$ is the projection of initial error onto the Chebyshev basis, $\kappa_0 = \pi$, and $\kappa_k = \pi/2$ for $k \geq 1$. Unlike MLPs with fixed activation functions, where high-frequency components typically have exponentially smaller eigenvalues [32], KANs maintain uniform eigenvalues $\eta \kappa_k = \eta \pi/2$ across all non-constant modes ($k \geq 1$). This enables simultaneous learning of both low- and high-frequency features, making them particularly well-suited for thermal field prediction.

IV. SCALING UP DEEPOHEAT VIA SEPARABLE TRAINING

Physics-informed training in DeepOHeat requires computing high-order spatial derivatives to evaluate residual and boundary terms in the loss function in (11). This necessitates AD, which applies the chain rule to compute exact derivatives through the neural network's computational graph, providing mathematically precise gradients essential for enforcing differential equation constraints.

However, applying AD in DeepOHeat presents significant computational challenges. For thermal simulation in 3D-ICs, the common-used reverse-mode AD typically employed in neural networks becomes prohibitively expensive when computing second-order derivatives across numerous collocation points. This results in excessive memory consumption and computational overhead that severely limit DeepOHeat's applicability to high-resolution thermal analysis.

To address the challenges in physics-informed training in DeepOHeat, we propose to decompose the spatial representation along the coordinate axes. This leads to a more efficient separable operator network [25], where the basis functions are learned independently for each spatial dimension.

A. Separable Representation for Single Points

For a given spatial coordinate $\mathbf{y} = (y_1, y_2, y_3)$ in a 3D-IC design, we approximate the temperature with a separable representation

$$G_{\theta}(\mathbf{u})(\mathbf{y}) = \sum_{k=1}^r \beta_k \prod_{n=1}^3 \tau_{n,k} = b_{\psi}(\mathcal{E}(\mathbf{u})) \cdot (t_{\phi_1}^1(y_1) \odot t_{\phi_2}^2(y_2) \odot t_{\phi_3}^3(y_3)). \quad (20)$$

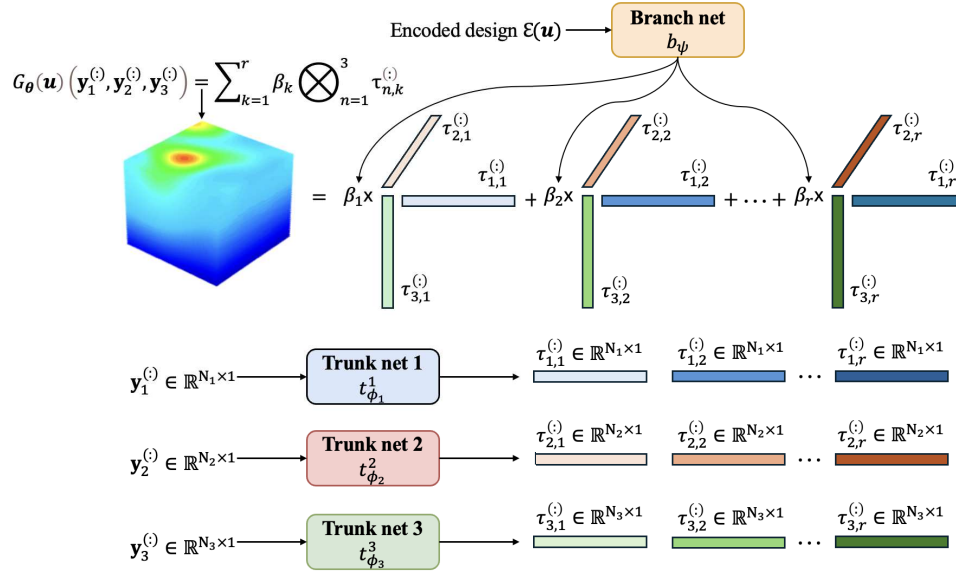


Fig. 4. Separable representation for thermal prediction in 3D-ICs. The model processes sets of points $\mathbf{y}_1^{(c)}$, $\mathbf{y}_2^{(c)}$, and $\mathbf{y}_3^{(c)}$ along each spatial dimension through three independent trunk networks to compute basis function values $\tau_{n,k}^{(c)}$. The branch network generates coefficients β_k that weight these basis functions, which are combined through outer products to efficiently represent the complete 3-D temperature field as a sum of rank-1 tensors.

Here $\odot$ denotes the Hadamard product.¹ Unlike DeepOHeat that uses a single trunk network, we employ three independent networks $t_{\phi_n}^n : \mathbb{R} \rightarrow \mathbb{R}^r$, each dedicated to learning basis functions $\tau_{n,k}$ for one spatial dimension. This separation allows for more efficient learning of spatial dependencies and enables parallel processing of spatial dimensions.

B. Grid-Based Evaluation

Consider a 3-D domain discretized with N_1 , N_2 , and N_3 points along each axis. Let $\mathbf{y}_n^{(c)} = \{y_n^{(1)}, y_n^{(2)}, \dots, y_n^{(N_n)}\}$ denote the set of coordinates along the n th axis. The complete meshgrid is formed by taking the Cartesian product of these coordinate sets, resulting in $N_1 \times N_2 \times N_3$ total points. We extend (20) to process this entire grid simultaneously

$$\begin{aligned} G_\theta(\mathbf{u})(\mathbf{y}_1^{(c)}, \mathbf{y}_2^{(c)}, \mathbf{y}_3^{(c)}) &= \sum_{k=1}^r \beta_k \bigotimes_{n=1}^3 \tau_{n,k}^{(c)} \\ &= \sum_{k=1}^r b_\psi(\mathcal{E}(\mathbf{u}))_k \left(t_{\phi_1}^1(\mathbf{y}_1^{(c)})_k \otimes t_{\phi_2}^2(\mathbf{y}_2^{(c)})_k \otimes t_{\phi_3}^3(\mathbf{y}_3^{(c)})_k \right) \end{aligned} \quad (21)$$

where $\otimes$ represents an outer product,² and $\tau_{n,k}^{(c)} = t_{\phi_n}^n(\mathbf{y}_n^{(c)})_k$ is the k th output of the n th trunk network evaluated at all points along the n th axis. As illustrated in Fig. 4, for each k , the outer product $\bigotimes_{n=1}^3 \tau_{n,k}^{(c)}$ produces a tensor of shape $N_1 \times N_2 \times N_3$ representing the contribution of the k th basis function across the entire grid. This decomposition enables efficient representation of the complete 3-D temperature distribution without evaluating each point individually.

¹The Hadamard product denotes the element-wise multiplication of vectors or matrices of the same dimensions.

²The outer product between vectors produces a higher-dimensional tensor whose entries are all possible products between elements of the original vectors.

C. Efficient Derivative Computation

Our separable structure, as visualized in Fig. 4, enables particularly efficient computation of spatial derivatives for three key reasons. First, as shown in the figure, the temperature field is decomposed into a sum of rank-1 tensors, each formed by outer products of basis functions from separate dimensions. This decomposition allows derivatives to be calculated independently along each dimension

$$\frac{\partial G_\theta(\mathbf{u})(\mathbf{y}_1^{(c)}, \mathbf{y}_2^{(c)}, \mathbf{y}_3^{(c)})}{\partial \mathbf{y}_m^{(c)}} = \sum_{k=1}^r \beta_k \left(\bigotimes_{n \neq m}^3 \tau_{n,k}^{(c)} \right) \otimes \frac{\partial t_{\phi_m}^m(\mathbf{y}_m^{(c)})_k}{\partial \mathbf{y}_m^{(c)}} \quad (22)$$

where $(\partial t_{\phi_m}^m(\mathbf{y}_m^{(c)})_k / \partial \mathbf{y}_m^{(c)})$ represents the derivative of the k th output of the m th trunk network with respect to its input coordinates.

Second, this formulation enables the use of forward-mode AD via Jacobian-vector products (JVPs), which is particularly efficient for our network architecture. In our separable structure, each trunk network $t_{\phi_m}^m : \mathbb{R}^{N_m} \rightarrow \mathbb{R}^{r \times N_m}$ takes N_m input coordinates and produces r basis functions evaluated at these coordinates. Forward-mode AD requires only $O(N_m)$ operations for JVP computation, while reverse-mode AD would require $O(r \times N_m)$ operations for vector-Jacobian products. Since r (typically 32–256) is much larger than 1 in our framework, forward-mode AD provides significant computational savings.

Third, as evident from Fig. 4, we can reuse the outputs $\tau_{n,k}^{(c)}$ from other dimensions ($n \neq m$) in the outer product computation, avoiding redundant calculations. This feature is particularly advantageous when evaluating higher-order or mixed derivatives, as each dimension's computation remains independent. In contrast, a non-separable architecture would require computing derivatives for each point in the

$N_1 \times N_2 \times N_3$ grid individually, leading to substantially higher computational complexity.

D. Overall Computational Benefit

The separable formulation offers significant advantages in physics-informed training of thermal prediction models. By allowing efficient derivative computation through forward-mode AD, our approach allows thermal analysis at higher resolutions than was previously possible with the original DeepOHeat architecture.

V. TRUSTWORTHY DESIGN OPTIMIZATION

In this section, we demonstrate how DeepOHeat-v1 can be integrated into a trustworthy design optimization framework for thermal optimization. By “trustworthy,” we mean predictions with quantifiable confidence and mechanisms for verification and refinement when accuracy is insufficient.

A. Thermal Optimization for Floorplan

Thermal optimization is critical in 3D-ICs due to increased power density and complex heat dissipation paths caused by vertical stacking. These thermal challenges directly impact device reliability, performance, and lifespan, making thermal management essential for viable 3-D integration technology.

The thermal optimization problem for 3D-IC floorplanning can be formulated mathematically as

$$\begin{aligned} \min_{\mathbf{u}} \quad & f(\mathbf{T}(\mathbf{u})) \\ \text{subject to} \quad & C_i(\mathbf{u}) \leq 0, \quad i = 1, 2, \dots, M \end{aligned} \quad (23)$$

where

$\mathbf{u}$	design configuration (component placement and properties);
$\mathbf{T}(\mathbf{u})$	temperature field for a given design $\mathbf{u}$;
$f(\mathbf{T}(\mathbf{u}))$	objective function, typically the peak temperature of the chip model;
$C_i(\mathbf{u}) \leq 0$	various design constraints.

As shown in the left part of Fig. 5, the optimization flow follows these general steps.

- 1) Initialize with a starting design $\mathbf{u}_{\text{ini}}$ and optimizer parameters.
- 2) For each iteration, generate new design candidates based on the current design.
- 3) Evaluate $\mathbf{T}(\mathbf{u})$ for each candidate design.
- 4) Calculate the objective function value $f(\mathbf{T}(\mathbf{u}))$.
- 5) Update the current design according to the optimizer.
- 6) Continue until termination criteria are met.

B. Confidence Estimation in Design Optimization

This optimization problem requires solving the heat equation for each design evaluation, making it computationally intensive. While operator learning models like DeepOHeat-v1 can significantly accelerate this process, they introduce new challenges for design reliability. Due to the closed-box nature of neural networks, there is no guarantee that the predicted result of the operator learning model is accurate

enough for every design case. In fact, DeepOHeat-v1 can have poor accuracy for some corner design cases. Consequently, an optimization solver may provide a low-quality solution once we use DeepOHeat-v1 directly in every optimization step.

In order to address this trustworthiness issue, we propose to assess the confidence of a DeepOHeat-v1 prediction at low cost and *without explicitly solving the underlying PDEs*. Specifically, assume that the PDE is first discretized (e.g., via an FD method) into a linear system

$$\mathbf{A}\mathbf{T} = \mathbf{b}. \quad (24)$$

Here, the system matrix $\mathbf{A}$ and load vector $\mathbf{b}$ are functions of the design configuration $\mathbf{u}$, which may include thermal conductivity variations, power distribution patterns, and BCs. We do not intend to solve for $\mathbf{b}$ due to the high computational cost. Instead, we can easily obtain $\mathbf{T}_0$, which is the predicted temperature at all discretization points using DeepOHeat-v1. The residual of the linear system provides a natural and efficient measure of solution quality

$$r = \frac{\|\mathbf{A}\mathbf{T}_0 - \mathbf{b}\|}{\|\mathbf{b}\|}. \quad (25)$$

This metric directly measures how well the predicted temperature field satisfies the discretized thermal equations without requiring the true solution. A small residual ($r < \alpha$) indicates that the prediction closely satisfies the governing equations, while a large residual suggests potential inaccuracies.

C. A Trustworthy Hybrid Optimization Framework

Based on the confidence estimator above, we now develop a hybrid optimization framework that combines the efficiency of operator learning with the reliability of numerical methods, as illustrated in Fig. 5. The right side of the figure shows our inner loop, which implements an adaptive solution strategy for each design evaluation.

- 1) For each new design configuration $\mathbf{u}$, obtain an initial prediction through the operator learning model

$$\mathbf{T}_0 = G_\theta(\mathbf{u}). \quad (26)$$

- 2) Evaluate prediction trustworthiness through the relative residual described in (25).
- 3) If $r < \alpha$, we accept $\mathbf{T}_0$ directly; otherwise, we refine the solution using GMRES with $\mathbf{T}_0$ as an *initial guess*. Although $\mathbf{T}_0$ is not accurate enough, it can play as an extremely high-quality initial guess of GMRES. Our implementation shows that this initial guess can reduce the number of GMRES iterations from thousands to a few dozen.

Only a small portion of the optimization steps need GMRES-based solution refinement. Furthermore, only a small number of GMRES iterations are required to solve each linear system. As a result, the entire optimization framework has a low computational cost, which is only marginally more expensive than using operator learning alone while providing the reliability of high-fidelity numerical methods.

In this work, we use simulated annealing [37] as the outer-loop optimizer (see Algorithm 1). However, our adaptive

Algorithm 1 Simulated Annealing for Thermal Optimization

Require: Initial design u_{ini} , initial SA temperature T_0 , cooling rate α , iterations N , base neighborhood size w_{base}

Ensure: Optimized design u^*

```

1:  $u_{curr} \leftarrow u_{ini}$ 
2:  $T \leftarrow T_0$ 
3:  $u^* \leftarrow u_{curr}$ 
4:  $f^* \leftarrow \text{EvaluateObjective}(u^*)$ 
5:  $f_{curr} \leftarrow f^*$ 
6: for  $i = 1$  to  $N$  do
7:   // Generate neighbor with adaptive neighborhood size
8:    $\text{temp\_ratio} \leftarrow T/T_0$ 
9:    $u_{new} \leftarrow u_{curr}$ 
10:  for each parameter  $p$  in  $u_{curr}$  do
11:     $\text{width} \leftarrow \max(1, \lfloor w_{base} \cdot \text{temp\_ratio} \rfloor)$ 
12:     $\text{low} \leftarrow \max(u_{curr}[p] - \text{width}, \text{param\_min}[p])$ 
13:     $\text{high} \leftarrow \min(u_{curr}[p] + \text{width}, \text{param\_max}[p])$ 
14:     $u_{new}[p] \leftarrow \text{RandomInteger}(\text{low}, \text{high})$ 
15:  end for
16:  // Evaluate objective function
17:   $f_{new} \leftarrow \text{EvaluateObjective}(u_{new})$ 
18:  // Accept or reject based on Metropolis criterion
19:  if  $f_{new} \leq f_{curr}$  or  $\exp((f_{curr} - f_{new})/T) > \text{random}(0, 1)$  then
20:     $u_{curr} \leftarrow u_{new}$ 
21:     $f_{curr} \leftarrow f_{new}$ 
22:    if  $f_{curr} < f^*$  then
23:       $u^* \leftarrow u_{curr}$ 
24:       $f^* \leftarrow f_{curr}$ 
25:    end if
26:  end if
27:  // Cool temperature
28:   $T \leftarrow \alpha \cdot T$ 
29: end for
30: return  $u^*$ 

```

solution method can be integrated with any other outer-loop optimizer, including but not limited to gradient descent, reinforcement learning, and Bayesian optimization.

VI. EXPERIMENTS

We evaluate our enhanced framework through comprehensive experiments, systematically demonstrating how each proposed improvement addresses the limitations of DeepOHeat. Our experiments use 1×1 mm chip dimensions in the xy-direction, following the original DeepOHeat paper for fair comparison, while our KAN and separable training improvements are architecture-independent and provide greater benefits at larger scales. First, we assess the framework on a 2-D power map thermal simulation benchmark to validate both prediction accuracy and computational efficiency improvements. Second, we validate our trustworthy optimization workflow in a floorplan design scenario, where reliable temperature prediction is crucial for finding optimal component placements. Throughout our experiments, we compare the following models.

TABLE I

PERFORMANCE COMPARISON FOR 2-D POWER MAP ON TOP SURFACE

Model	MAPE (%)	Training Time (h)	GPU Memory (GB)
DeepOHeat	0.060	0.53	17.44
DeepOHeat + KAN	0.048	0.51	17.32
DeepOHeat-v1	0.049	0.0084	0.56

- 1) *DeepOHeat*: The baseline operator learning model from [1] using MLP-based trunk networks.
- 2) *DeepOHeat + KAN*: An enhanced variant that replaces the MLP in the trunk network with KANs for improved representational capacity.
- 3) *DeepOHeat-v1*: Our fully enhanced model implementing both KAN-based trunk nets and separable training.
- 4) *DeepOHeat + ST*: An ablation variant incorporating only our separable training approach with the original MLP architecture (introduced in Section VI-B).

A. 2-D Power Map on the Top Surface

1) *Problem Formulation and Implementation*: Following DeepOHeat [1], we consider a single-cuboid chip model with dimensions $1 \times 1 \times 0.5$ mm, discretized on a $21 \times 21 \times 11$ mesh grid (Fig. 6). The power map is defined on a 20×20 tiling of the top surface, with each tile's power level scaled by 0.00625 mW and interpolated onto the 21×21 computational grid. We maintain adiabatic conditions on the side surfaces and convection conditions on the bottom surface ($\text{HTC} = 500 \text{ W/m}^2 \cdot \text{K}$, $T_{\text{amb}} = 298.15 \text{ K}$), with homogeneous thermal conductivity $k = 0.1 \text{ W/mK}$.

All models share the same branch network architecture (9 layers, 256 neurons per layer), processing the 441-D flattened power map input. For trunk networks, the baseline DeepOHeat employs a single six-layer MLP (128 nodes/layer) with Fourier feature mapping ($\mu = 0$, $\sigma = 2\pi$) and swish activation. DeepOHeat + KAN maintains the single trunk network architecture but replaces the MLP with a four-layer order-3 Chebyshev KAN (64 nodes/layer) to keep the parameter budget comparable. DeepOHeat-v1 implements our separable training approach with three independent trunk networks (one per spatial dimension), each having the same KAN architecture as DeepOHeat + KAN. The physics-informed loss is evaluated on the full computational mesh. To generate diverse training data, we sample 50 power maps per iteration from a 2-D standard Gaussian random field (GRF) with a length scale parameter of 0.3. All implementations are built with JAX for efficient computation. Models are trained for 10 000 iterations using ADAM optimizer (learning rate $1e^{-3}$, decayed by $0.9 \times$ every 500 iterations) on an NVIDIA RTX 3090 GPU.

2) *Results Analysis*: Table I presents performance comparisons with $N_f = 50$ sampled functions per iteration. The reported mean absolute percentage error (MAPE) values are calculated and averaged based on ten representative power maps from the original DeepOHeat paper. Appendix provides visualizations of all ten test cases. Importantly, these evaluation power maps consist of structured heat blocks that differ significantly from the GRF patterns used during training.

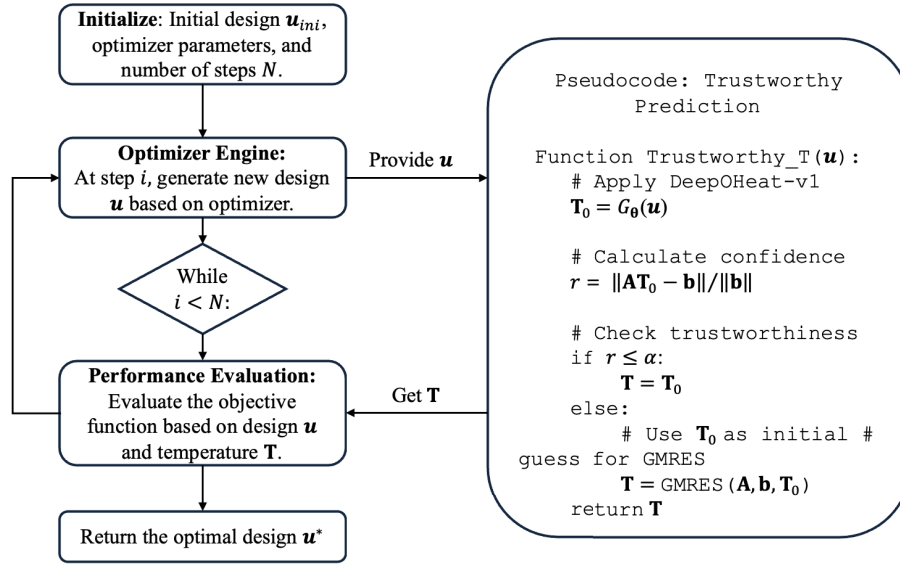


Fig. 5. Hybrid thermal optimization framework for 3D-IC design. The framework consists of two nested loops: 1) an outer optimizer loop (left) that explores the design space using any optimization algorithm of choice; and 2) an inner trustworthy prediction loop (right) that ensures reliable thermal evaluation by applying DeepOHeat-v1 and adaptively refining predictions with GMRES when the residual exceeds a predefined threshold.

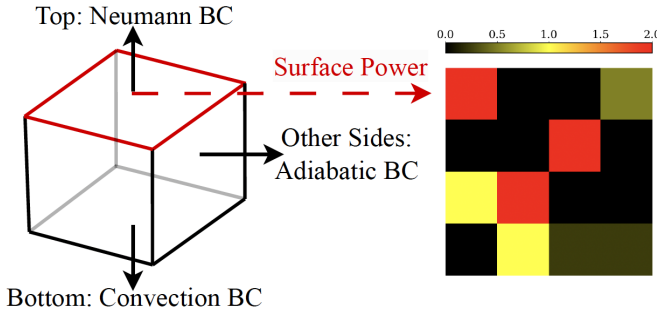


Fig. 6. Chip geometry used in thermal simulations. Left: Conceptual single-cuboid model with isotropic material properties. Right: Power map applied to the top surface.

DeepOHeat + KAN demonstrates superior prediction accuracy, reducing MAPE from 0.060% to 0.048% compared to the baseline. This $1.25\times$ reduction in error validates that KAN's learnable activation functions better capture multiscale thermal patterns. The computational cost remains comparable to the baseline (0.51 versus 0.53 h, 17.32 versus 17.44 GB), indicating the improved accuracy comes without additional overhead.

DeepOHeat-v1 maintains the enhanced prediction capability (0.049% MAPE) while drastically reducing computational requirements through separable training. Training time decreases by $62\times$ (from 0.51 h to 30.24 s) and memory usage by $31\times$ (from 17.32 to 0.56 GB), demonstrating effective mitigation of the computational bottleneck.

Fig. 7 illustrates the practical impact of these improvements with one representative multihotspot power distribution from our test set. Visual comparison between the Celsius 3-D reference solution and model predictions reveals that DeepOHeat struggles with accurately capturing sharp temperature gradients around hotspots, particularly in the central and corner regions. In contrast, DeepOHeat-v1 produces temperature

fields that more closely match the reference solution, with better-defined hotspot shapes and intensities. This demonstrates KAN's superior ability to represent the multiscale features inherent in thermal fields, particularly when combined with our separable training approach.

Fig. 8 further analyzes model behavior across varying N_f values, revealing several key insights.

- 1) *Architectural Impact*: DeepOHeat + KAN consistently achieves lower MAPE than the baseline across all N_f values, with the improvement becoming more pronounced at larger batch sizes. This demonstrates KAN's robust advantage in capturing thermal patterns.
- 2) *Training Efficiency*: Although both baseline and KAN-only variants show rapid growth in computational requirements with increasing N_f , DeepOHeat-v1 maintains nearly constant memory usage (0.56 GB) and sub-minute training time. This scalability is crucial for practical applications that require larger training batches.
- 3) *Synergistic Benefits*: DeepOHeat-v1 combines the accuracy of KAN with the efficiency of separable training, achieving optimal performance in all metrics.

B. Volumetric Power Distribution in Three-Layer Architecture

In this section, we evaluate our framework in a more challenging thermal simulation scenario involving volumetric power distributions in a complex 3-D chip architecture. This experiment presents a significant computational challenge: high-resolution 3-D thermal analysis with volumetric power distribution across multiple materials.

1) *Problem Setup*: We consider a three-layer chip model ($1 \times 1 \times 0.55$ mm) discretized on a $101 \times 101 \times 56$ computational mesh (Fig. 9). The model has thermal conductivities of 1 W/mK (top layer, 0.4 mm), 1 W/mK (middle layer,

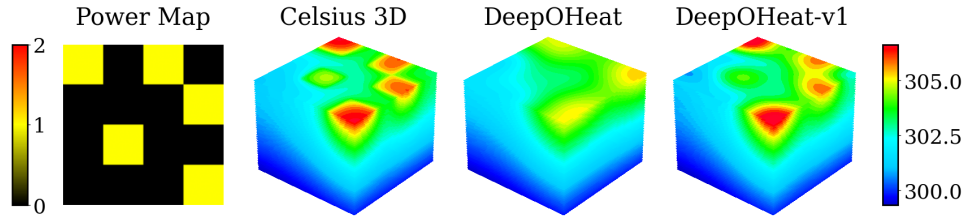


Fig. 7. Comparison of temperature field predictions for a representative multihotspot power map. From left to right: Input power map showing the complex thermal pattern, reference solution by Celsius 3-D, temperature predictions by DeepOHeat, and temperature predictions by DeepOHeat-v1. The visualization demonstrates DeepOHeat-v1's superior ability to capture multiscale thermal features, particularly at hotspot regions where the baseline DeepOHeat model fails to capture more detailed patterns.

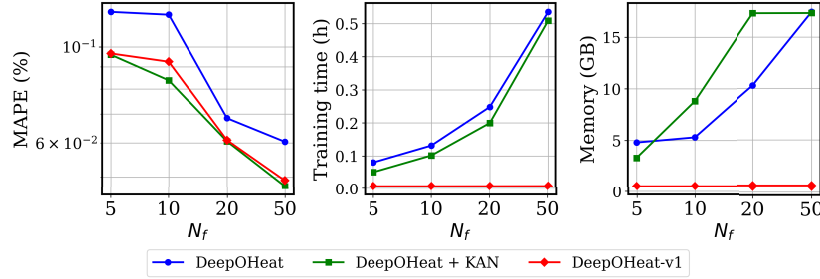


Fig. 8. Performance comparison of three models in terms of MAPE, GPU memory usage, and training time for different N_f .

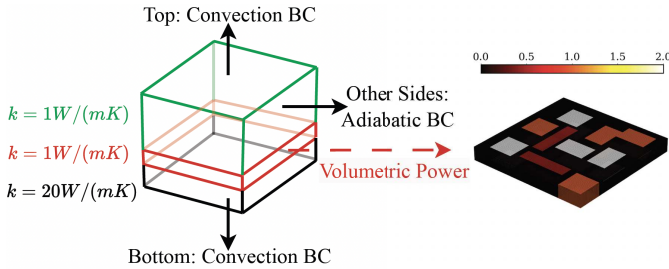


Fig. 9. 3-D chip geometry: (left) schematic showing top (green), middle (red), and bottom (gray) layers; and (right) a volumetric power map (heat sources) applied to the middle layer.

0.05 mm), and 20 W/mK (bottom layer, 0.1 mm), with adiabatic side boundaries and convection on the top/bottom ($HTC = 500 \text{ W/m}^2 \cdot \text{K}$, $T_{\text{amb}} = 298.15 \text{ K}$). The power distribution is defined by ten rectangular components (C0–C9) placed in the middle layer. Each component occupies a fixed area (400 tiles on a 100×100 grid) but with varying power: 0.5 mW (C0, C1), 1.0 mW (C2–C5), and 2.0 mW (C6–C9). As shown in Fig. 10, different spatial arrangements of these components result in distinct volumetric power patterns, creating diverse thermal loading scenarios.

2) *Surrogate Model Training and Evaluation*: The high-resolution mesh ($101 \times 101 \times 56$), representing a spatial resolution of approximately $10 \mu\text{m}$ per grid point, presents a significant computational challenge, requiring more than 570 K spatial points during the physics-informed training. During training, we minimize the physics loss across this entire fine mesh, enforcing thermal governing equations and BCs at high spatial resolution. This high-resolution physics-informed training produces more accurate surrogate models compared to training on coarser grids, and was computationally infeasible

TABLE II

PERFORMANCE COMPARISON OF DEEPOHEAT-V1 VARIANTS FOR VOLUMETRIC POWER IN THREE-LAYER CHIP ARCHITECTURE

Model	MAPE (%)	Training Time (h)	GPU Memory (GB)
DeepOHeat + ST	0.22	0.50	12.09
DeepOHeat-v1	0.035	0.52	12.10

with the original DeepOHeat due to memory limitations. Only models incorporating separable training—DeepOHeat + ST and DeepOHeat-v1—can be effectively trained on an NVIDIA RTX 3090 GPU.

We maintain the same network architectures as in Section VI-A, with the only difference being the branch network input dimension increased to 10201 to handle the 101×101 power map in the middle layer. During training, we generate random floorplan configurations by stochastically placing the ten components, allowing overlap for efficient sampling. Each model is trained for 100 000 iterations using ADAM optimizer (learning rate $1e^{-3}$, decayed by $0.9 \times$ every 1000 iterations), with 50 floorplans sampled per iteration. The physics-informed loss enforces thermal governing equations and BCs across the entire computational domain.

For performance evaluation, we generate 100 random test floorplans following the same sampling procedure as the training data. The prediction accuracy is measured against reference solutions computed using the GMRES with algebraic multigrid preconditioning. Table II presents a detailed performance comparison. DeepOHeat-v1 achieves dramatically improved accuracy, reducing MAPE from 0.22% to 0.035%—a $6.29 \times$ reduction in error—while maintaining similar computational efficiency (approximately 0.5 h of training time and 12 GB memory usage). This significant improvement highlights KAN's superior capability in capturing complex 3-

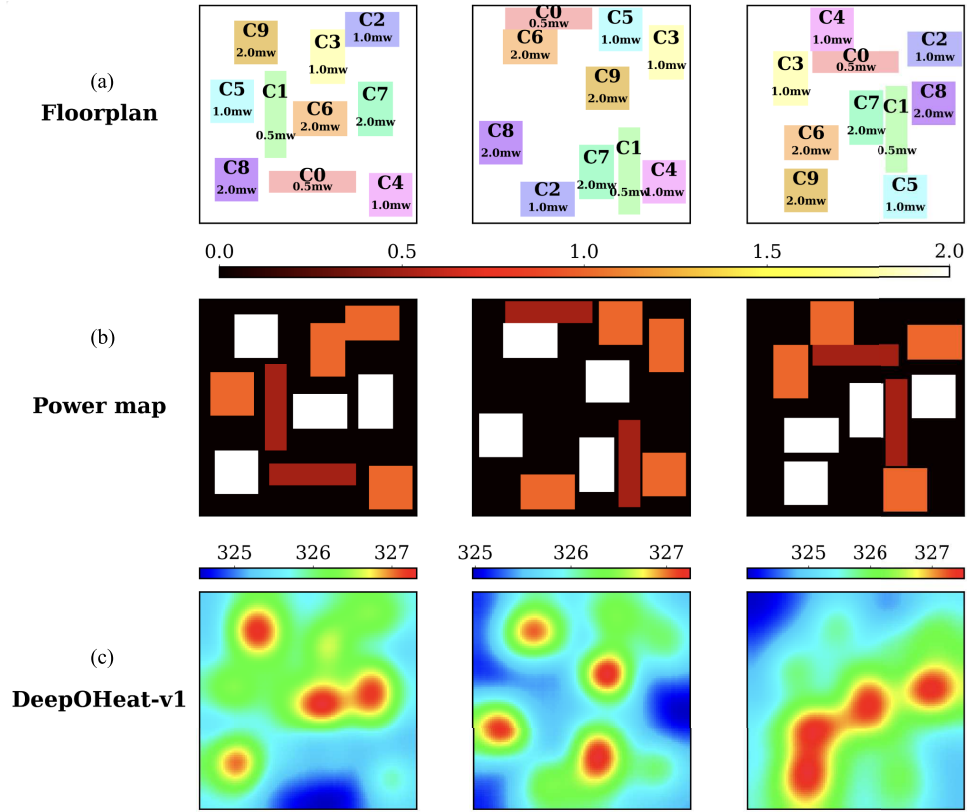


Fig. 10. (a)–(c) Three example floorplan configurations and their thermal responses. Top row: spatial layout of ten components (C0–C9) with different power densities; middle row: resulting power maps on the computational grid; and bottom row: temperature distributions predicted by DeepOHeat-v1 at the 13th vertical slice ($z = 0.12$ mm).

D thermal patterns, while separable training enables scalability to high resolutions.

Fig. 10 shows example temperature predictions for three different component arrangements. The temperature distributions at the 13th vertical slice ($z = 0.12$ mm) show the formation of hotspots at locations with high-power components (e.g., C6–C9), and thermal coupling between neighboring components.

C. Thermal Optimization for Floorplan

Finally, we use our DeepOHeat-v1 to optimize the floorplan of the three-layer design described in Section VI-B. We first evaluate our confidence estimator and solution refinement method. Then we evaluate the performance of the entire floorplan optimizer.

1) *Confidence Estimator and Solution Refinement*: We evaluate three solution methods for thermal analysis using the left configuration of Fig. 10.

- 1) GMRES attempts to solve the thermal linear system directly from a random initial guess, demonstrating the challenge of the thermal problem and the importance of a good initial guess for convergence.
- 2) The operator learning model (DeepOHeat-v1) provides direct temperature predictions through the trained operator model without solving the linear system.
- 3) Our hybrid approach first obtains an initial prediction from DeepOHeat-v1, and then refines it using GMRES

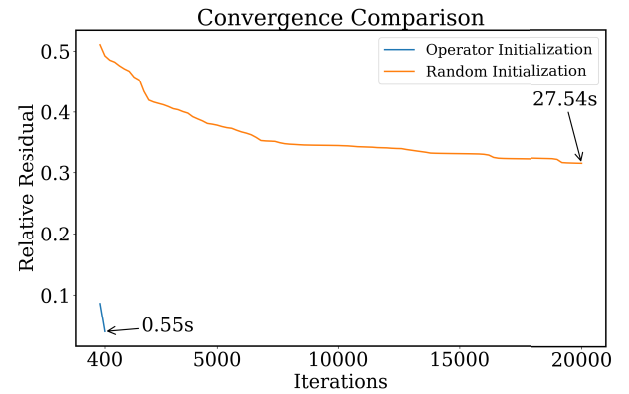


Fig. 11. Convergence comparison between GMRES with different initialization strategies. The operator initialization (blue) leverages DeepOHeat-v1 predictions as the initial guess, enabling rapid convergence in 0.55 s. Random initialization (orange) requires 27.54 s and still fails to reach acceptable accuracy after 20 000 iterations.

when the prediction residual exceeds the threshold. This shows how a good initial guess from the operator model significantly accelerates GMRES convergence.

All GMRES implementations utilize CuPy [38] for GPU acceleration, with parameters set to: restart = 200, maximum iterations = 20 000, and tolerance = 0.05. Fig. 11 compares the convergence behavior of GMRES with different initializations. Standard GMRES with a random initial guess converges

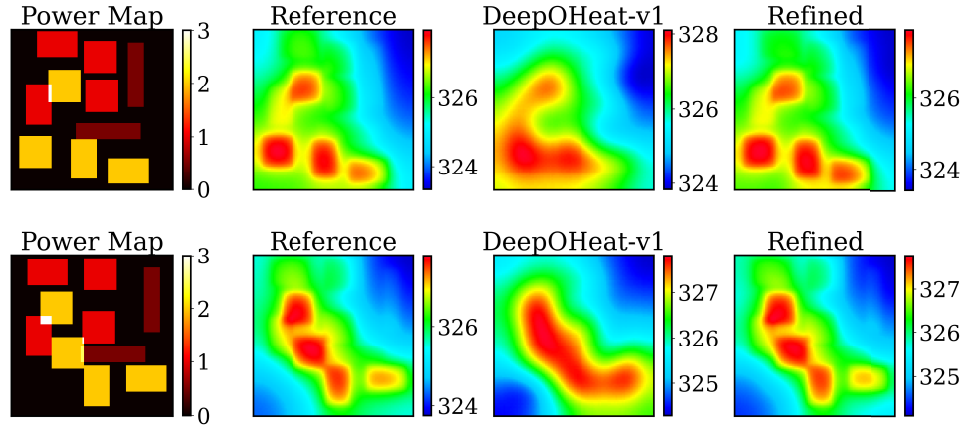


Fig. 12. Thermal prediction reliability assessment during optimization. Top and bottom rows show two cases where operator model predictions required refinement. For each case: power distribution (left), reference solution (second), DeepOHeat-v1 prediction (third), and hybrid method refinement (right). The refined solutions correct the inaccuracies in the neural network predictions.

slowly, taking 27.54 s to reach a modest residual level even after 20 000 iterations. In contrast, when initialized with our operator learning prediction, GMRES converges rapidly to a relative residual of 0.04 in just 0.55 s. This $50\times$ speedup demonstrates the effectiveness of neural network predictions as initial guesses for iterative solvers.

Fig. 12 shows two representative cases where the predictions from operator learning require refinement during optimization. While the operator model captures general thermal patterns, the hybrid refinement successfully corrects local prediction errors to match the reference solution. This is particularly evident in areas with closely positioned high-power components, where thermal coupling effects create complex temperature gradients that are challenging to predict perfectly.

2) *Overall Performance of Thermal Optimization for Floorplan*: We evaluate our hybrid framework in a thermal optimization task in floorplanning. The optimization objective is to minimize the peak temperature of the chip model by determining optimal component placements while avoiding overlaps through adding a soft penalty term to the objective function. We implement simulated annealing using Optuna [39] with 1000 optimization steps, an initial SA temperature of 0.5, a cooling rate of 0.997, and a base neighborhood size of 25 (as detailed in Algorithm 1). For the hybrid framework's residual threshold, we set α to 9.

Fig. 13 shows the distribution of relative residual values across all design evaluations in our optimization process. With our selected threshold of $\alpha = 9$ (indicated by the vertical dashed line), we observe that approximately 30% of the design evaluations triggered the GMRES refinement step. This analysis confirms that while most predictions from DeepOHeat-v1 were sufficiently accurate, a significant portion required additional refinement to ensure solution reliability.

Table III compares the three thermal evaluation approaches. The results demonstrate the effectiveness of our hybrid framework in balancing accuracy and efficiency. GMRES with algebraic multigrid preconditioning serves as our reference, providing the most accurate solution but at the highest computational cost. Pure operator learning achieves a significant

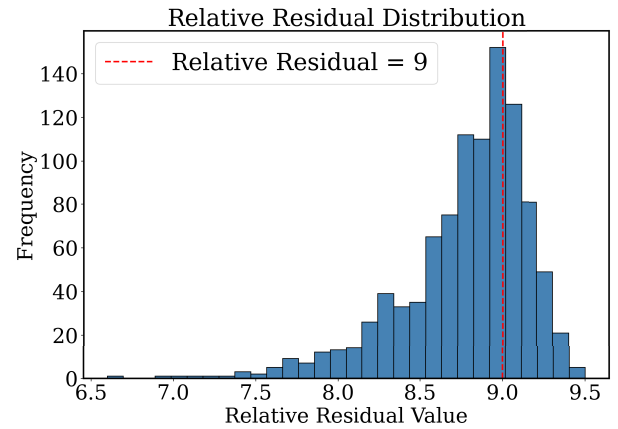


Fig. 13. Distribution of relative residual values from DeepOHeat-v1 predictions across 1000 optimization trials.

TABLE III
FLOORPLAN OPTIMIZATION PERFORMANCE ANALYSIS USING DIFFERENT THERMAL EVALUATION METHODS

Evaluator in Optimization	Max Temp (K)	Opt. Time (h)
GMRES w/ preconditioning	327.18	4.94
Operator learning only	328.50	0.05
Proposed hybrid solver	327.31	0.07

speedup (0.05 versus 4.94 h) but does not offer a guarantee of trustworthiness during the optimization process. Our hybrid approach addresses this concern by monitoring the quality of the prediction through residual verification, triggering non-preconditioned GMRES refinement in 299 out of 1000 optimization steps. By ensuring reliable temperature evaluation throughout the optimization, the hybrid approach not only provides solution guarantees but also achieves better design quality (327.31 versus 328.50 K maximum temperature) compared to pure operator learning, while closely matching the reference solution (327.18 K). This improvement comes with minimal computational overhead (0.02 h additional optimization time), demonstrating the value of selective refinement for trustworthy design optimization.

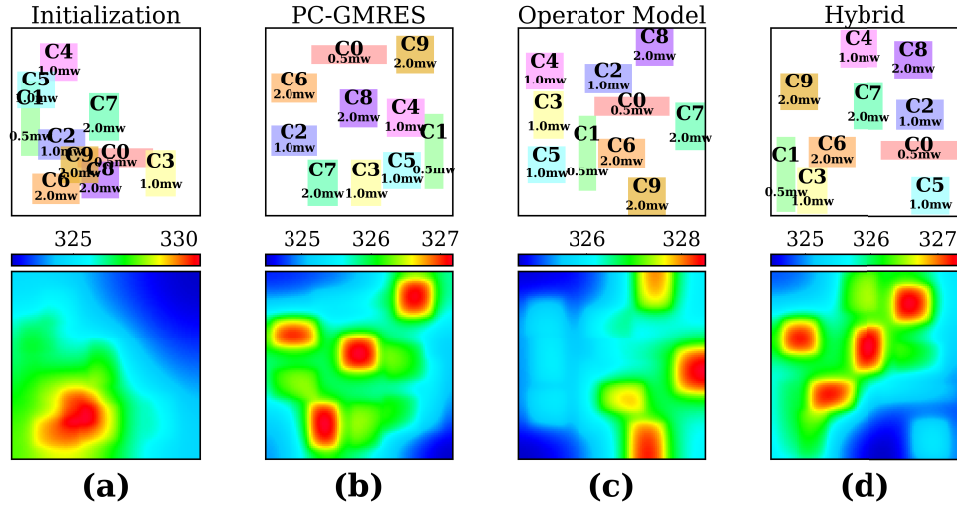


Fig. 14. Floorplan configurations (top) and their corresponding temperature distributions (bottom). (a) Initial design before optimization. (b) Optimized design using PC-GMRES with algebraic multigrid preconditioning. (c) Optimized design using the operator model only. (d) Optimized design using a hybrid approach.

The final optimized floorplans in Fig. 14 show that our hybrid approach produces design solutions of similar quality to non-machine learning methods while reducing computing time by $70\times$.

VII. CONCLUSION

This work has presented DeepOHeat-v1 to significantly enhance the original DeepOHeat framework through three key innovations. By replacing MLP trunk networks with KANs, we have enabled adaptive learning of multiscale thermal features, leading to substantial improvements in prediction accuracy for both surface and volumetric power distributions. The employment of separable training has overcome the computational bottleneck in physics-informed training, making high-resolution thermal analysis feasible on standard GPU hardware. Most importantly, our confidence estimator and hybrid optimization framework transform operator learning from a fast but potentially unreliable predictor into a trustworthy design optimization tool for thermal optimization in 3D-IC design. The selective refinement strategy effectively balances efficiency and reliability, achieving accuracy comparable to an optimization engine using high-fidelity reference mesh-based PDE solvers while obtaining a significant speedup. This capability is particularly valuable for practical 3D-IC design, where reliable thermal evaluation is crucial to finding optimal designs in design space exploration and signal/power integrity sign-off.

These advances open new possibilities for efficient thermal optimization in increasingly complex 3D-IC and chiplet design. Several important extensions can build upon this foundation. First, extending to transient thermal analysis by incorporating time as an additional input dimension to the trunk networks. Second, addressing full 3D-IC specific challenges, including through-silicon via (TSV) thermal modeling, inter-die thermal coupling, and thermal-aware placement considering packaging constraints. Third, validation on larger chip sizes representative of modern designs. Fourth, and critically,

our current test cases demonstrate modest temperature gradients; validation on modern high-performance designs with extreme gradients is essential to fully assess the method's capabilities. Finally, extending our framework to address more practical thermal-aware floorplan optimization problems, including considerations for power delivery networks, timing constraints, and multiobjective optimization scenarios that balance thermal performance with other critical design metrics.

APPENDIX

A. Visualization of All Test Cases for 2-D Surface Power Map

Fig. 15 shows the complete set of ten test cases used to compute the average MAPE values reported in Table I. The error maps reveal that improvements are minimal for simple cases (columns 1–3) but become significant for complex multihotspot patterns (columns 4–10), with DeepOHeat-v1 particularly excelling at maintaining accuracy near thermal boundaries.

B. Detailed Derivation of Training Dynamics and Error Evolution

The target function $f^*(y)$ on $y \in [-1, 1]$ is assumed to have a Chebyshev series expansion

$$f^*(y) = \sum_{k=0}^{\infty} b_k C_k(y) \quad (27)$$

where $\{C_k(y)\}$ are Chebyshev polynomials. We train a single-layer KAN model

$$f(y) = \sum_{k=0}^K a_k C_k(y) \quad (28)$$

by minimizing the weighted L^2 loss

$$\mathcal{L} = \frac{1}{2} \int_{-1}^1 (f(y) - f^*(y))^2 \frac{dy}{\sqrt{1-y^2}}. \quad (29)$$

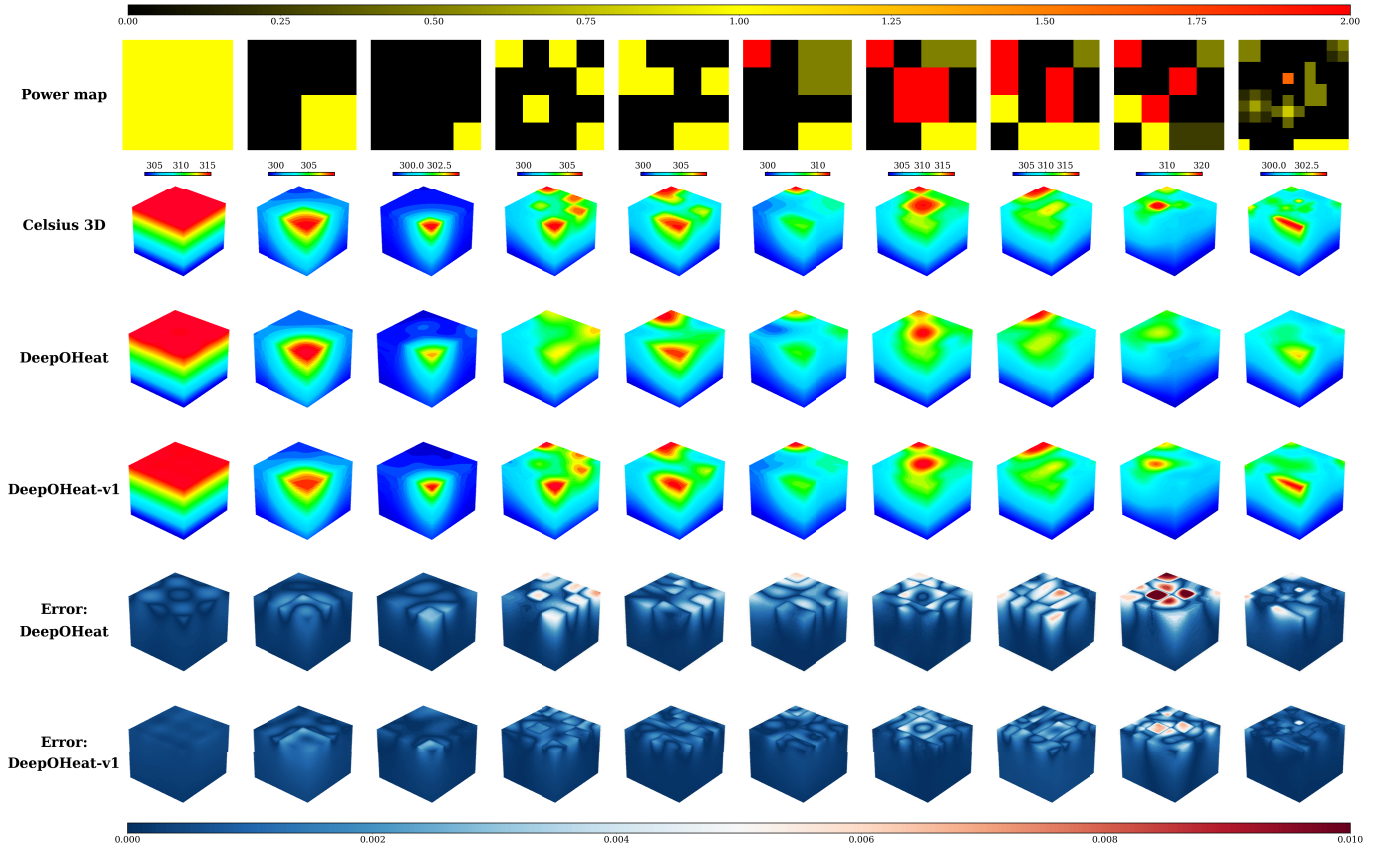


Fig. 15. All ten test power maps from the original DeepOHeat paper used in Section VI-A evaluation. Top row: power distributions. Rows 2–4: Temperature predictions from Celsius 3-D (reference), DeepOHeat, and DeepOHeat-v1. Bottom rows: Error maps for DeepOHeat and DeepOHeat-v1.

The gradient of $\mathcal{L}$ with respect to parameter a_k is

$$\frac{\partial \mathcal{L}}{\partial a_k} = \int_{-1}^1 (f(y) - f^*(y)) C_k(y) \frac{dy}{\sqrt{1-y^2}}. \quad (30)$$

Under continuous-time gradient descent (gradient flow), parameters evolve as

$$\begin{aligned} \frac{da_k(t)}{dt} &= -\eta \frac{\partial \mathcal{L}}{\partial a_k} \\ &= -\eta \int_{-1}^1 (f(y, t) - f^*(y)) C_k(y) \frac{dy}{\sqrt{1-y^2}} \end{aligned} \quad (31)$$

where $\eta > 0$ is the learning rate. The time derivative of the network output $f(y, t) = \sum_{k=0}^K a_k(t) C_k(y)$ becomes

$$\begin{aligned} \frac{\partial f(y, t)}{\partial t} &= \sum_{k=0}^K \frac{da_k(t)}{dt} C_k(y) \\ &= -\eta \int_{-1}^1 \left[\sum_{k=0}^K C_k(y) C_k(y') \right] \\ &\quad \times (f(y', t) - f^*(y')) \frac{dy'}{\sqrt{1-y'^2}}. \end{aligned} \quad (32)$$

This defines the NTK

$$\Theta_{\text{KAN}}(y, y') = \sum_{k=0}^K C_k(y) C_k(y'). \quad (33)$$

Define the error $e(y, t) = f(y, t) - f^*(y)$. Projecting $e(y, t)$ onto the Chebyshev basis

$$e(y, t) = \sum_{k=0}^K e_k(t) C_k(y) \quad (34)$$

where the coefficients $e_k(t)$ are computed via

$$e_k(t) = \frac{1}{\kappa_k} \int_{-1}^1 e(y, t) C_k(y) \frac{dy}{\sqrt{1-y^2}} \quad (35)$$

and the normalization constants κ_k arise from Chebyshev orthogonality

$$\int_{-1}^1 C_i(y) C_j(y) \frac{dy}{\sqrt{1-y^2}} = \kappa_i \delta_{ij} \quad (36)$$

where $\kappa_0 = \pi$, $\kappa_k = (\pi/2)$ ($k \geq 1$).

Substitute $e(y, t) = \sum_{k=0}^K e_k(t) C_k(y)$ into the gradient flow equation (32)

$$\begin{aligned} \frac{\partial e(y, t)}{\partial t} &= -\eta \int_{-1}^1 \Theta_{\text{KAN}}(y, y') e(y', t) \frac{dy'}{\sqrt{1-y'^2}} \\ &= -\eta \sum_{k=0}^K \kappa_k e_k(t) C_k(y). \end{aligned} \quad (37)$$

Equating coefficients for each $C_k(y)$, we obtain decoupled ODEs

$$\frac{de_k(t)}{dt} = -\eta \kappa_k e_k(t) \quad (38)$$

with solutions

$$e_k(t) = e_k(0) e^{-\eta_k t}. \quad (39)$$

The total error evolution is therefore

$$e(y, t) = \sum_{k=0}^K e_k(0) e^{-\eta_k t} C_k(y). \quad (40)$$

This contrasts with MLPs, where NTK eigenvalues typically decay exponentially with frequency [35], causing high-frequency modes to converge slower. KANs' uniform spectral convergence enables simultaneous learning of multiscale features.

REFERENCES

- [1] Z. Liu et al., "DeepOHeat: Operator learning-based ultra-fast thermal simulation in 3D-IC design," in *Proc. 60th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2023, pp. 1–6.
- [2] J. H. Lau, "TSV manufacturing yield and hidden costs for 3D IC integration," in *Proc. 60th Electron. Compon. Technol. Conf. (ECTC)*, Jun. 2010, pp. 1031–1042.
- [3] B. Wu and A. Kumar, "Extreme ultraviolet lithography and three dimensional integrated circuit—A review," *Appl. Phys. Rev.*, vol. 1, no. 1, Mar. 2014, Art. no. 011104.
- [4] H. Delaram, A. Dastfan, and M. Norouzi, "Optimal thermal placement and loss estimation for power electronic modules," *IEEE Trans. Compon., Packag., Manuf. Technol.*, vol. 8, no. 2, pp. 236–243, Feb. 2018.
- [5] K. Cao, J. Zhou, T. Wei, M. Chen, S. Hu, and K. Li, "A survey of optimization techniques for thermal-aware 3D processors," *J. Syst. Archit.*, vol. 97, pp. 397–415, Aug. 2019.
- [6] H. Sultan, A. Chauhan, and S. R. Sarangi, "A survey of chip-level thermal simulators," *ACM Comput. Surv.*, vol. 52, no. 2, pp. 1–35, Mar. 2020.
- [7] F. Tavakkoli, S. Ebrahimi, S. Wang, and K. Vafai, "Analysis of critical thermal issues in 3D integrated circuits," *Int. J. Heat Mass Transf.*, vol. 97, pp. 337–352, Jun. 2016.
- [8] Z. Liu, S. Swarup, S. X.-D. Tan, H.-B. Chen, and H. Wang, "Compact lateral thermal resistance model of TSVs for fast finite-difference based thermal analysis of 3-D stacked ICs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 33, no. 10, pp. 1490–1502, Oct. 2014.
- [9] P. Li, L. T. Pileggi, M. Asheghi, and R. Chandra, "Efficient full-chip thermal modeling and analysis," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Sep. 2004, pp. 319–326.
- [10] T.-Y. Wang and C. C.-P. Chen, "SPICE-compatible thermal simulation with lumped circuit modeling for thermal reliability analysis based on modeling order reduction," in *Proc. Int. Symp. Signals, Circuits Syst. (SCS)*, 2004, pp. 357–362.
- [11] J. Xie and M. Swaminathan, "System-level thermal modeling using nonconformal domain decomposition and model-order reduction," *IEEE Trans. Compon., Packag., Manuf. Technol.*, vol. 4, no. 1, pp. 66–76, Jan. 2014.
- [12] J. Wen et al., "DNN-based fast static on-chip thermal solver," in *Proc. 36th Semiconductor Thermal Meas., Model. Manage. Symp. (SEMI-THERM)*, Mar. 2020, pp. 65–75.
- [13] A. Kumar et al., "ML-based fast on-chip transient thermal simulation for heterogeneous 2.5D/3D IC designs," in *Proc. Int. Symp. VLSI Design, Autom. Test (VLSI-DAT)*, Apr. 2022, pp. 1–8.
- [14] R. Ranade, H. He, J. Pathak, N. Chang, A. Kumar, and J. Wen, "A thermal machine learning solver for chip simulation," in *Proc. ACM/IEEE 4th Workshop Mach. Learn. CAD (MLCAD)*, Sep. 2022, pp. 111–117.
- [15] D. Geb et al., "On-chip transient hot spot detection with a multiscale ROM in 3DIC designs," in *Proc. IEEE 72nd Electron. Compon. Technol. Conf. (ECTC)*, May 2022, pp. 221–232.
- [16] M. Wang, Y. Cheng, W. Zeng, Z. Lu, V. F. Pavlidis, and W. Xing, "ARO: Autoregressive operator learning for transferable and multi-fidelity 3D-IC thermal analysis with active learning," in *Proc. 43rd IEEE/ACM Int. Conf. Computer-Aided Design*, Oct. 2024, pp. 1–9.
- [17] M. J. Smith et al., "Real-time precision prediction of 3-D package thermal maps via image-to-image translation," in *Proc. IEEE 32nd Conf. Electr. Perform. Electron. Packag. Syst. (EPEPS)*, Oct. 2023, pp. 1–3.
- [18] H. He and J. Pathak, "An unsupervised learning approach to solving heat equations on chip based on auto encoder and image gradient," 2020, *arXiv:2007.09684*.
- [19] H. Sanchis-Alepuz and M. Stipsitz, "Towards real time thermal simulations for design optimization using graph neural networks," in *Proc. IEEE Design Methodologies Conf. (DMC)*, Sep. 2022, pp. 1–6.
- [20] Z. Lu, Y. Zhou, Y. Zhang, X. Hu, Q. Zhao, and X. Hu, "A fast general thermal simulation model based on multi-branch physics-informed deep operator neural network," *Phys. Fluids*, vol. 36, no. 3, Mar. 2024.
- [21] M. Tancik et al., "Fourier features let networks learn high frequency functions in low dimensional domains," in *Proc. Adv. Neural Inf. Process. Syst.*, Jun. 2020, pp. 7537–7547.
- [22] S. Wang, H. Wang, and P. Perdikaris, "On the eigenvector bias of Fourier feature networks: From regression to solving multi-scale PDEs with physics-informed neural networks," *Comput. Methods Appl. Mech. Eng.*, vol. 384, Oct. 2021, Art. no. 113938.
- [23] Z. Liu et al., "KAN: Kolmogorov–Arnold networks," 2024, *arXiv:2404.19756*.
- [24] S. S. Sidharth, A. R. Keerthana, R. Gokul, and K. P. Anas, "Chebyshev polynomial-based Kolmogorov–Arnold networks: An efficient architecture for nonlinear function approximation," 2024, *arXiv:2405.07200*.
- [25] X. Yu et al., "Separable operator networks," *Trans. Mach. Learn. Res.*, vol. 2024, 2024. [Online]. Available: <https://openreview.net/forum?id=RYJmFDXv>
- [26] K. A. Khan and P. I. Barton, "A vector forward mode of automatic differentiation for generalized derivative evaluation," *Optim. Methods Softw.*, vol. 30, no. 6, pp. 1185–1212, Nov. 2015.
- [27] Y. Saad and M. H. Schultz, "GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems," *SIAM J. Sci. Stat. Comput.*, vol. 7, no. 3, pp. 856–869, Jul. 1986.
- [28] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, "Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators," *Nature Mach. Intell.*, vol. 3, no. 3, pp. 218–229, Mar. 2021.
- [29] P. Jin, S. Meng, and L. Lu, "MIONet: Learning multiple-input operators via tensor product," *SIAM J. Sci. Comput.*, vol. 44, no. 6, pp. A3490–A3514, Dec. 2022.
- [30] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *J. Comput. Phys.*, vol. 378, pp. 686–707, Feb. 2019.
- [31] S. Wang, H. Wang, and P. Perdikaris, "Learning the solution operator of parametric partial differential equations with physics-informed DeepONets," *Sci. Adv.*, vol. 7, no. 40, p. 8605, Oct. 2021.
- [32] R. Basri, D. Jacobs, Y. Kasten, and S. Kritchman, "The convergence rate of neural networks for learned functions of different frequencies," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 4761–4771.
- [33] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: A survey," *J. Mach. Learn. Res.*, vol. 18, no. 153, pp. 1–43, 2018.
- [34] J. Schmidt-Hieber, "The Kolmogorov–Arnold representation theorem revisited," *Neural Netw.*, vol. 137, pp. 119–126, May 2021.
- [35] A. Jacot, F. Gabriel, and C. Hongler, "Neural tangent kernel: Convergence and generalization in neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 8580–8589.
- [36] J. Lee et al., "Wide neural networks of any depth evolve as linear models under gradient descent," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 8572–8583.
- [37] P. J. Van Laarhoven, E. H. Aarts, P. J. van Laarhoven, and E. H. Aarts, *Simulated Annealing*. Cham, Switzerland: Springer, 1987.
- [38] R. Nishino and S. H. C. Loomis, "Cupy: A numpy-compatible library for NVIDIA GPU calculations," in *Proc. 31st Conf. Neural Inf. Process. Syst.*, 2017, vol. 151, no. 7.
- [39] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, 2019, pp. 2623–2631.